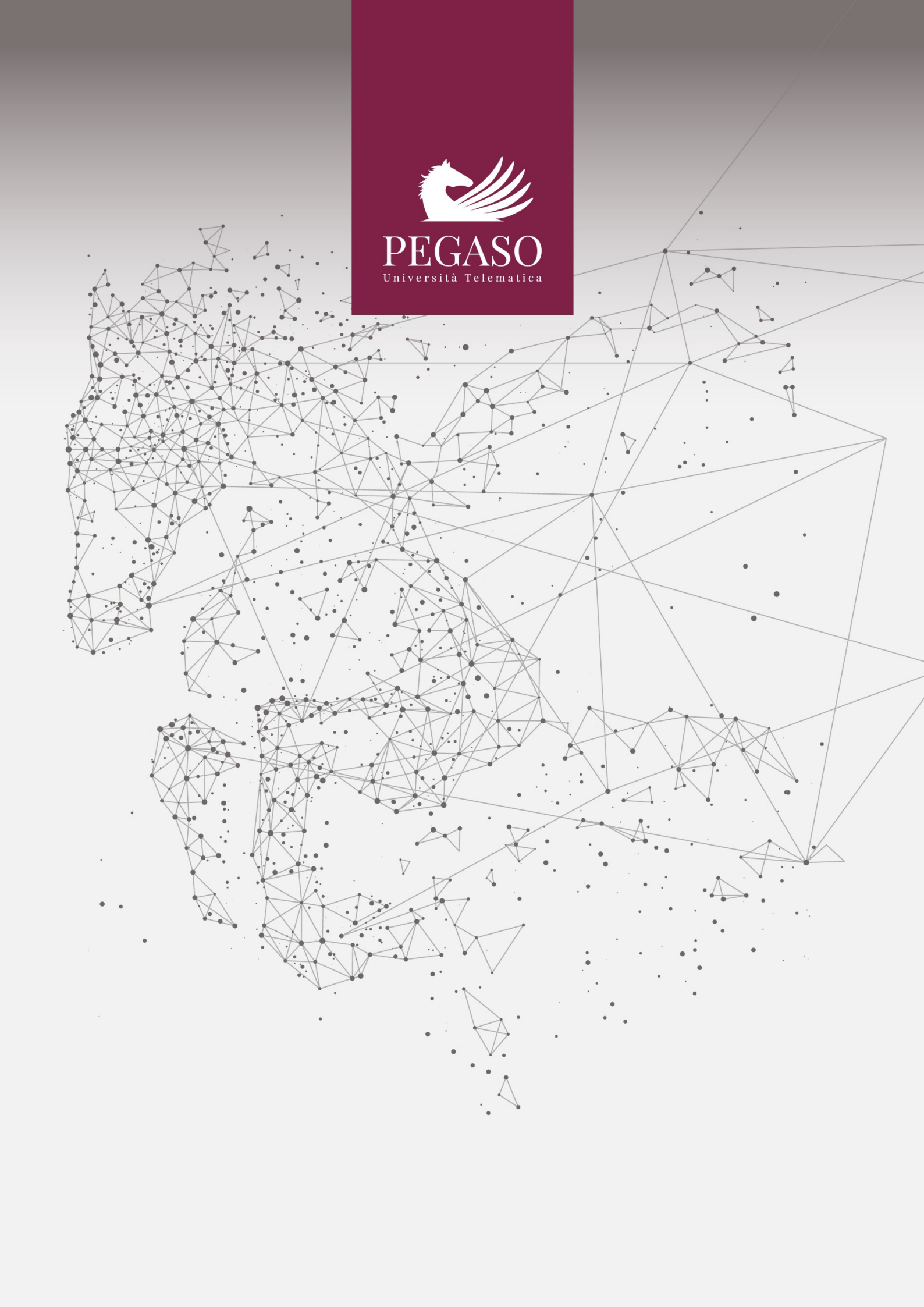

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. PRODOTTO MATRICIALE 4
3. MATRICE BERSAGLIO 9
4. GIRELLA DI NUMERI 15
BIBLIOGRAFIA 21

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano ed analizzano alcune esercitazioni avanzate che approfondiscono la progettazione delle strutture iterative per il caricamento e l'elaborazione delle matrici (liste bidimensionali). In particolare, si riportano tre esercizi avanzati in cui si risolvono problemi riferiti alle matrici. Il primo esercizio mostra come effettuare il prodotto tra matrici. Il secondo esercizio mostra come impostare gli intervalli di variabilità degli indici per suddividere una matrice quadrata in strati concentrici. Il terzo esercizio mostra come impostare gli intervalli di variabilità degli indici per caricare una matrice quadrata secondo uno schema predeterminato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Prodotto matriciale

In algebra lineare, il prodotto matriciale (o moltiplicazione di matrici) è il prodotto righe per colonne tra due matrici, possibile sotto certe condizioni, che dà luogo ad un'altra matrice. Se una matrice rappresenta una applicazione lineare, il prodotto fra matrici è la traduzione della composizione di due applicazioni lineari.

Il prodotto matriciale ha diverse proprietà:

- generalmente non è commutativa - es. AB ≠ BA;
- distributiva rispetto alla somma - es. A(B+C) = AB + AC;
- per ogni scalare k vale k(AB) = (kA)B = A(kB);
- associativa - es. A(BC) = (AB)C;
- elemento neutro è la matrice identità *I* (matrice avente tutti gli elementi a zero, tranne quelli della diagonale pari a 1).

Per poter operare il prodotto matriciale AB tra la matrice A (di dimensioni k × z) e la matrice B (di dimensioni x × y) deve essere rispettata la seguente condizione: z=x, cioè il numero delle colonne della prima matrice fattore deve essere uguale al numero di righe della seconda matrice fattore ad es. A(2 × 3) e B(3 × 2).

Sia C = AB (con A e B riportate di seguito), l'elemento in posizione (1,1) di c sarà uguale alla sommatoria dei prodotti $a_{11}*b_{11}$, $a_{12}*b_{21}$ e $a_{13}*b_{31}$:

$$c_{11} = 1*4 + 0*(-2) + 2*0 = 4$$

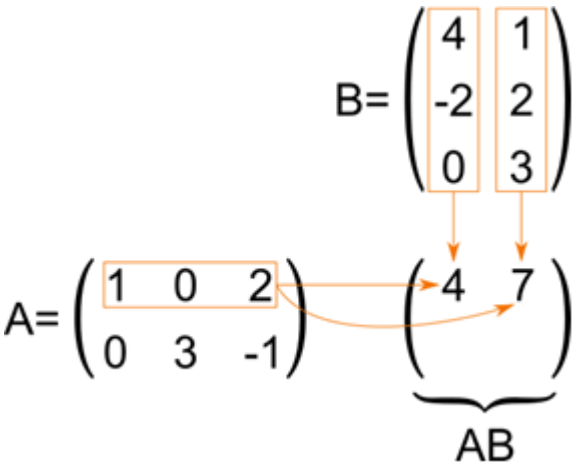
$$A = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 3 & -1 \end{pmatrix} \qquad B = \begin{pmatrix} 4 & 1 \\ -2 & 2 \\ 0 & 3 \end{pmatrix}$$

$$C = AB = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per quanto riguarda gli altri elementi di C, come schematicamente riportato di seguito:

- l'elemento c_{12} sarà la sommatoria dei prodotti degli elementi della prima riga di A e della seconda colonna di B;
- l'elemento c_{21} sarà la sommatoria dei prodotti degli elementi della seconda riga di A e della prima colonna di B;
- l'elemento c_{22} sarà la sommatoria dei prodotti degli elementi della seconda riga di A e della seconda colonna di B.



Si propone un esempio completo con il prodotto tra A(3 × 3) e B(3 × 2):

$$AB = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 5 & -1 \\ 3 & 2 & 0 \end{pmatrix} \begin{pmatrix} 7 & 1 \\ 1 & 0 \\ 0 & 4 \end{pmatrix} =$$

$$= \begin{pmatrix} 1\cdot 7+0\cdot 1+1\cdot 0 & 1\cdot 1+0\cdot 0+1\cdot 4 \\ 1\cdot 7+5\cdot 1+(-1)\cdot 0 & 1\cdot 1+5\cdot 0+(-1)\cdot 4 \\ 3\cdot 7+2\cdot 1+0\cdot 0 & 3\cdot 1+2\cdot 0+0\cdot 4 \end{pmatrix} = \begin{pmatrix} 7 & 5 \\ 12 & -3 \\ 23 & 3 \end{pmatrix}$$

Il main(), riportato di seguito, richiede tre parametri (r1 il numero di righe della matrice A, c il numero delle colonne della matrice A che deve essere uguale al numero delle righe della matrice B, c2 il

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

numero delle colonne della matrice B). Attraverso la funzione carica_matrici(), che verrà esaminata a breve, ottiene le matrici A e B caricate con numeri casuali. Si noti il commento inserito per le matrici A e B: per poter testare l'algoritmo è possibile inserire delle matrici di prova. Successivamente, vengono visualizzate le matrici A e B, e viene calcolato il prodotto matriciale AB per ottenere la matrice risultato ris con la funzione prodotto_matrici(). Infine, viene visualizzata la matrice ris.

```
def main():
    print("\nPRODOTTO MATRICIALE AB\n")
    r1 = int(input("Inserire numero di righe della matrice A: "))
    c = int(input("Inserire numero di colonne della matrice A = righe matrice B: "))
    c2 = int(input("Inserire numero di colonne della matrice B: "))
    a,b=carica_matrici(r1,c,c2)

    #a = [[1,0,1],[1,5,-1],[3,2,0]]
    #b = [[7,1],[1,0],[0,4]]
    print("\nMatrice A")
    visualizza_matrice(a)
    print("\nMatrice B")
    visualizza_matrice(b)
    ris = prodotto_matrici(a,b)
    print("\n\nMatrice C=AB")
    visualizza_matrice(ris)


main()
```

Si riporta il codice delle funzioni carica_matrici() e visualizza_matrice(). La funzione carica_matrici() che necessita della libreria random, opportunamente importata all'inizio del codice, richiede in ingresso 3 parametri: r1 il numero di righe della matrice A, c il numero delle colonne della matrice A che deve essere uguale al numero delle righe della matrice B, c2 il numero delle colonne della matrice B. Per una migliore comprensibilità del codice, si pongono sia c1 (numero di colonne di A) sia r2 (numero di righe di B) uguali a c. Vengono, quindi, inizializzate le due matrici, ognuna con le proprie dimensioni, con numeri casuali interi compresi tra 1 e 15.

La funzione visualizza_matrice() che richiede come parametro la matrice da visualizzare, desume il numero di righe e di colonne della matrice e ne permette la visualizzazione a schermo con una formattazione opportuna.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import random

def carica_matrici(r1,c,c2):
    c1=c
    r2=c
    a = [0]*r1
    for i in range(r1):
        riga=[0]*c1
        a[i] = riga
    for i in range(r1):
        for j in range(c1):
            a[i][j]=random.randint(1,15)

    b = [0]*r2
    for i in range(r2):
        riga=[0]*c2
        b[i] = riga
    for i in range(r2):
        for j in range(c2):
            b[i][j]=random.randint(1,15)

    return a,b

def visualizza_matrice(m):
    righe = len(m)
    colonne = len(m[0])
    for i in range(righe):
        for j in range(colonne):
            print(m[i][j],end="\t")
        print("")
```

L'ultima funzione del programma è la funzione prodotto_matrici() che richiede come parametri le due matrici da moltiplicare ed effettua il vero e proprio prodotto matriciale. In particolare, la funzione desume le dimensioni delle due matrici (r1,c1,r2,c2). Successivamente, inizializza la matrice risultato ad una matrice nulla avente come dimensioni r1 x c2. Gli elementi nulli serviranno pari a zero perché fungeranno da accumulatori per i prodotti.

Attraverso due strutture iterative annidate impostate per coprire tutti gli elementi della matrice risultato c (i per le righe da 0 a r1-1 e j per le colonne da 0 a c2-1), è possibile impostare una terza struttura iterativa annidata il cui indice k varia tra 0 e r2-1 (o c1-1 che è uguale) che permette il prodotto degli elementi delle due matrici e la somma nel corrispondente elemento di c che funge da accumulatore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def prodotto_matrici(a,b):
    r1 = len(a)
    c1 = len(a[0])
    r2 = len(b)
    c2 = len(b[0])

    # si imposta tabella risultante c=ab di dim r1 x c2
    c = [0]*r1
    for i in range(r1):
        riga=[0]*c2
        c[i] = riga

    #prodotto matriciale
    for i in range(r1):      #si itera per ciascuna riga di c
       for j in range(c2):   #si itera per ciascuna colonna di c
           for k in range(r2):
               c[i][j] += a[i][k] * b[k][j]
    return c
```

```
PRODOTTO MATRICIALE AB

Inserire numero di righe della matrice A: 3
Inserire numero di colonne della matrice A = righe matrice B: 4
Inserire numero di colonne della matrice B: 5

Matrice A
7       4       12      7
12      1       14      9
11      5       5       14

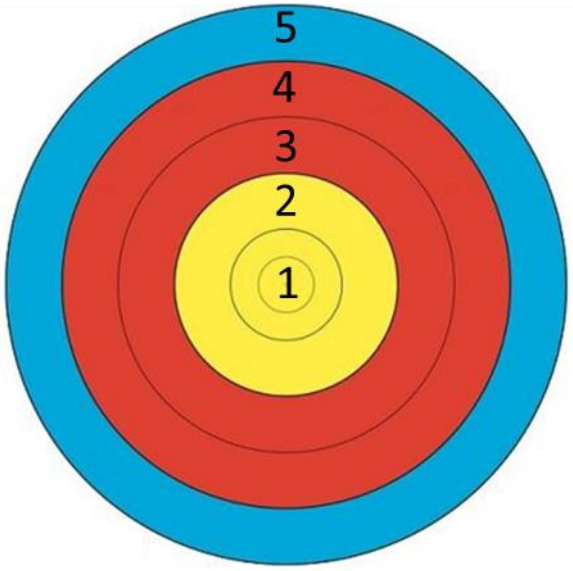
Matrice B
9       10      4       5       2
7       8       5       1       10
12      9       13      4       1
9       13      7       13      12


Matrice C=AB
298     301     253     178     150
364     371     298     234     156
320     377     232     262     245
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Matrice bersaglio

L'esercizio che si vuole risolvere non ha applicazioni immediate, ma permette di approfondire la pianificazione degli intervalli di variabilità degli indici delle strutture iterative per l'inserimento e per l'elaborazione delle liste. Si vuole risolvere il problema del bersaglio al contrario con le matrici come schematizzato nella figura seguente partendo da 1 al centro del bersaglio con numeri crescenti allontanandosi dal centro del bersaglio stesso.



In particolare, si vuole risolvere il seguente problema:

- data una matrice quadrata di dimensione *n* con *n* dispari;
- si parte dall'elemento centrale della matrice assegnandogli valore 1;
- ci si sposta di «cerchio» in «cerchio» (di «strato» in «strato») verso l'esterno della matrice aumentando il valore di ciascun elemento di un'unità;
- al termine la matrice sarà riempita in modo da formare degli strati "concentrici" con valore crescente dal centro verso l'esterno.

  L'obiettivo è ottenere il seguente risultato per una matrice quadrata di esempio di dimensioni 7x7.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
4   4   4   4   4   4   4
4   3   3   3   3   3   4
4   3   2   2   2   3   4
4   3   2   1   2   3   4
4   3   2   2   2   3   4
4   3   3   3   3   3   4
4   4   4   4   4   4   4
```

Prima di procedere con la progettazione del codice della funzione che permette di caricare gli elementi della matrice secondo lo schema del bersaglio, si analizza il main() e la funzione di visualizzazione della matrice. Il main() è molto semplice poiché chiede, attraverso un ciclo di validazione dell'input, un numero intero dispari (che sarà la dimensione della matrice quadrata) e lo passa come parametro alla funzione di caricamento della matrice.

La funzione visualizza_matrice(), che prende come parametro la matrice da visualizzare, analizza le dimensioni della matrice stessa e permette la visualizzazione corretta della matrice.

Di seguito si riporta il codice delle due funzioni.

```
def visualizza_matrice(m):
    righe = len(m)
    colonne = len(m[0])
    for i in range(righe):
        for j in range(colonne):
            print(m[i][j],end="\t")
        print("")

def main():
    print("\nMATRICE BERSAGLIO\n")
    n=0
    while n%2==0:
        n = int(input("Inserire numero di righe//colonne della matrice quadrata: "))
    m=carica_matrice_bers(n)
    visualizza_matrice(m)


main()
```

La funzione carica_matrice_bers() prende come input la dimensione della matrice quadrata *n* (con *n* dispari). Successivamente, la funzione opera nel seguente modo:

1. crea una matrice nulla di dimensione *nxn*;
2. crea una matrice booleana con elementi tutti a False che tiene traccia degli elementi già inseriti nella matrice stessa ed evita la sovrascrittura dei valori. L'approccio che utilizza la matrice booleana corrispondente è facilmente semplificabile modificando l'algoritmo (un algoritmo alternativo che

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

inserisce i numeri dall'esterno della matrice verso l'interno della stessa non prevede l'utilizzo della matrice booleana). Ad ogni modo, in questo esercizio si vuole mostrare volutamente una tecnica di risoluzione degli esercizi con una matrice booleana detta anche «maschera» che fornisce informazioni supplementari sulla matrice che si sta costruendo. Segue il codice di questi primi due passaggi.

```
def carica_matrice_bers(n):
    m = [0]*n
    for i in range(n):
        riga=[0]*n
        m[i] = riga

    b = [False]*n
    for i in range(n):
        riga=[False]*n
        b[i] = riga
```

Successivamente, la funzione opera nel seguente modo:

3. procede all'inserimento dell'elemento centrale pari a 1 e si pone a True il corrispondente elemento nella matrice booleana b;
4. la variabile k terrà traccia del numero dello strato che per ciascuna iterazione si vuole inserire;
5. tale valore viene inizializzato a 2 poiché il primo strato (un solo elemento centrale è stato già assegnato). Segue il codice dei passaggi 3,4,5:

```
i=n//2
j=n//2
m[i][j]=1              #elemento centrale
b[i][j]=True

k=2
```

6. la funzione imposta, poi, un ciclo while che permette di adattare il numero degli strati da inserire in base alla dimensione n della matrice quadrata. All'interno di questo ciclo si impostano due strutture iterative annidate che permettono di «spazzare», per ciascuna iterazione di tale ciclo, un'area della matrice sempre maggiore;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

7. all'interno dei tre cicli annidati, una struttura condizionale permette di verificare se l'elemento della matrice booleana b è stato già inizializzato al valore corrispondente allo strato;
8. solo se l'elemento di b è False, nella matrice m, al corrispondente elemento viene assegnato il valore dello strato. Per maggiore chiarezza, al termine del ciclo while più esterno, oltre ad incrementare il valore di k, si visualizza la matrice booleana per capire come lo strato si accresce. Di seguito si riporta la porzione di codice relativa ai punti 6,7, e 8.

```
while n//2-k+1>=0:
    for i in range(n//2-k+1,n//2+k):
        for j in range(n//2-k+1,n//2+k):
            if not(b[i][j]):
                m[i][j]=k
                b[i][j]=True
    k+=1
    visualizza_matrice(b)
    print("")

return m
```

Di seguito si riporta il codice completo della funzione carica_matrice_bers().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def carica_matrice_bers(n):
    m = [0]*n
    for i in range(n):
        riga=[0]*n
        m[i] = riga

    b = [False]*n
    for i in range(n):
        riga=[False]*n
        b[i] = riga

    i=n//2
    j=n//2
    m[i][j]=1           #elemento centrale
    b[i][j]=True

    k=2
    while n//2-k+1>=0:
        for i in range(n//2-k+1,n//2+k):
            for j in range(n//2-k+1,n//2+k):
                if not(b[i][j]):
                    m[i][j]=k
                    b[i][j]=True
        k+=1
        visualizza_matrice(b)
        print("")

    return m
```

Si riporta, ora, il risultato dell'elaborazione del codice Python completo dell'esercizio "Matrice bersaglio" per una dimensione della matrice quadrata n pari a 7.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
MATRICE BERSAGLIO

Inserire numero di righe//colonne della matrice quadrata: 7
False   False   False   False   False   False   False
False   False   False   False   False   False   False
False   False   True    True    True    False   False
False   False   True    True    True    False   False
False   False   True    True    True    False   False
False   False   False   False   False   False   False
False   False   False   False   False   False   False

False   False   False   False   False   False   False
False   True    True    True    True    True    False
False   True    True    True    True    True    False
False   True    True    True    True    True    False
False   True    True    True    True    True    False
False   True    True    True    True    True    False
False   False   False   False   False   False   False

True    True    True    True    True    True    True
True    True    True    True    True    True    True
True    True    True    True    True    True    True
True    True    True    True    True    True    True
True    True    True    True    True    True    True
True    True    True    True    True    True    True
True    True    True    True    True    True    True

4       4       4       4       4       4       4
4       3       3       3       3       3       4
4       3       2       2       2       3       4
4       3       2       1       2       3       4
4       3       2       2       2       3       4
4       3       3       3       3       3       4
4       4       4       4       4       4       4
```

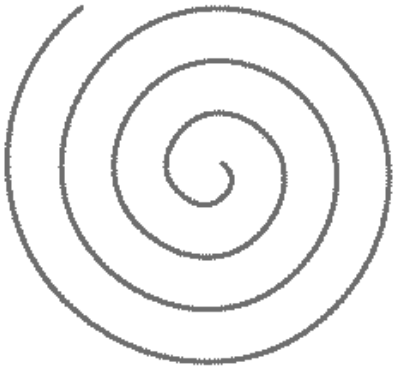
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Girella di numeri

L'esercizio che si vuole risolvere non ha applicazioni immediate, ma permette di approfondire la pianificazione degli intervalli di variabilità degli indici delle strutture iterative per l'inserimento e per l'elaborazione delle liste. Si vuole risolvere il problema della girella di numeri con le matrici.

Data una matrice quadrata di lato n (con n dispari), si vuole inserire in tale matrice un numero progressivo che parte da 1 (al centro della matrice) e, spostandosi nella matrice «girando» in senso orario, si vuole riempire la matrice arrivando fino al valore *n*n* come schematizzato nella figura seguente.



Prima di procedere con la progettazione del codice della funzione che permette di caricare gli elementi della matrice secondo lo schema della «girella», si analizza il main() e la funzione di visualizzazione della matrice.

Il main() è molto semplice poiché chiede, attraverso un ciclo di validazione dell'input un numero intero dispari (che sarà la dimensione della matrice quadrata) e lo passa come parametro alla funzione di caricamento della matrice.

La funzione visualizza_matrice(), che prende come parametro la matrice da visualizzare, analizza le dimensioni della matrice stessa e permette la visualizzazione corretta della matrice.

Di seguito si riporta il codice Python delle due funzioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def visualizza_matrice(m):
    righe = len(m)
    colonne = len(m[0])
    for i in range(righe):
        for j in range(colonne):
            print(m[i][j],end="\t")
        print("")


def main():
    print("\nGIRELLA DI NUMERI\n")
    n=0
    while n%2==0:
        n = int(input("Inserire numero di righe//colonne della matrice quadrata: "))
    m=carica_matrice_gir(n)
    visualizza_matrice(m)


main()
```

La funzione carica_matrice_gir() prende come input la dimensione della matrice quadrata n (con n dispari). Successivamente, la funzione opera nel seguente modo:

1. crea una matrice nulla m di dimensione nxn;
2. calcola gli indici dell'elemento centrale (pari a n//2) e procede all'inserimento dell'elemento centrale pari a 1;
3. la variabile k terrà traccia del numero dello strato che si sta considerando (parte da 2 poiché l'elemento centrale corrisponde allo strato 1);
4. la variabile progr è la variabile «progressivo» che si incrementa ogni volta che nella matrice viene inserito un elemento. Di seguito si riporta la porzione di codice della funzione che implementa i punti da 1 a 4.

```
def carica_matrice_gir(n):

    m = [0]*n
    for i in range(n):
        riga=[0]*n
        m[i] = riga

    ic=n//2
    jc=n//2
    m[ic][jc]=1            #elemento centrale

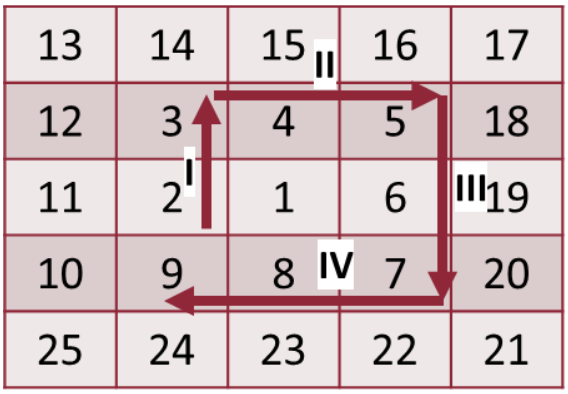
    progr=2
    k=2
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Anche se il problema sembra di semplice risoluzione, la progettazione dell'algoritmo di inserimento degli elementi nella matrice risulta essere complessa. Per semplificare la risoluzione di tale esercizio, che si ricorda deve valere per qualsiasi n dispari, si scompone l'inizializzazione degli elementi in quattro parti distinte come schematicamente riportato nella figura seguente:

1. la prima parte prevede l'inserimento degli elementi a sinistra dal basso verso l'alto;
2. la seconda parte prevede l'inserimento degli elementi in alto da sinistra verso destra;
3. la terza parte prevede l'inserimento degli elementi a destra dall'alto verso il basso;
4. la quarta parte prevede l'inserimento degli elementi in basso da destra verso sinistra.



Il codice imposta un ciclo while che permette di adattare il numero degli strati in cui inserire i numeri in base alla dimensione n della matrice quadrata. Ciascuna freccia nello schema corrisponde ad un ciclo for all'interno del ciclo while più esterno come riportato nella porzione di codice Python della funzione carica_matrice_gir() che segue. Sulla sinistra delle strutture iterative è indicato, con numeri romani, la corrispondente "linea" di elementi della matrice che si sta valorizzando.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
while n//2-k+1>=0:
    for i in range(ic+k-2,ic-k,-1):  #lato sinistro verso l'alto
        m[i][jc-k+1]=progr                                          I
        progr+=1

    for j in range(jc-k+2,jc+k):     #lato superiore verso destra
        m[ic-k+1][j]=progr                                          II
        progr+=1

    for i in range(ic-k+2,ic+k):     #lato destro verso il basso
        m[i][jc+k-1]=progr
        progr+=1                                                    III

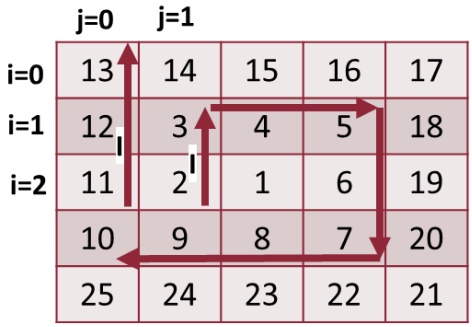
    for j in range(jc+k-2,jc-k,-1):  #lato inferiore verso sinistra
        m[ic+k-1][j]=progr                                          IV
        progr+=1

    k+=1

return m
```

Come è facilmente intuibile osservando lo schema, le frecce (e di conseguenza il numero di elementi corrispondenti nella matrice) aumenteranno di dimensioni all'aumentare dello strato (cioè allontanandosi dall'elemento centrale con indici ic e jc). Per tale motivo, gli intervalli di variazione degli indici dovranno dipendere dal valore dello strato k.

Per una migliore comprensione dell'esercizio, si riporta la tabella di traccia del primo ciclo for (l'operazione può essere ripetuta anche per gli altri cicli for) valida, come schematicamente riportato nella figura sotto per una matrice quadrata di dimensioni 5x5.



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

n	ic	jc	k	ic+k-2	ic-k	jc-k+1
5	2	2	2	2	0	1
5	2	2	3	3	-1	0

La tabella di traccia appena riportata permette di visualizzare l'intervallo di variazione degli indici i e j all'interno della matrice.

Di seguito si riporta il codice Python completo della funzione carica_matrice_gir().

```
def carica_matrice_gir(n):

    m = [0]*n
    for i in range(n):
        riga=[0]*n
        m[i] = riga

    ic=n//2
    jc=n//2
    m[ic][jc]=1           #elemento centrale

    progr=2
    k=2

    while n//2-k+1>=0:
        for i in range(ic+k-2,ic-k,-1):  #lato sinistro verso l'alto
            m[i][jc-k+1]=progr
            progr+=1

        for j in range(jc-k+2,jc+k):     #lato superiore verso destra
            m[ic-k+1][j]=progr
            progr+=1

        for i in range(ic-k+2,ic+k):     #lato destro verso il basso
            m[i][jc+k-1]=progr
            progr+=1

        for j in range(jc+k-2,jc-k,-1):  #lato inferiore verso sinistra
            m[ic+k-1][j]=progr
            progr+=1

        k+=1

    return m
```

Si riporta, ora, il risultato dell'elaborazione del codice completo del programma, per un errore volontario di n pari (n=6) e, successivamente, per una matrice di dimensioni 7x7.

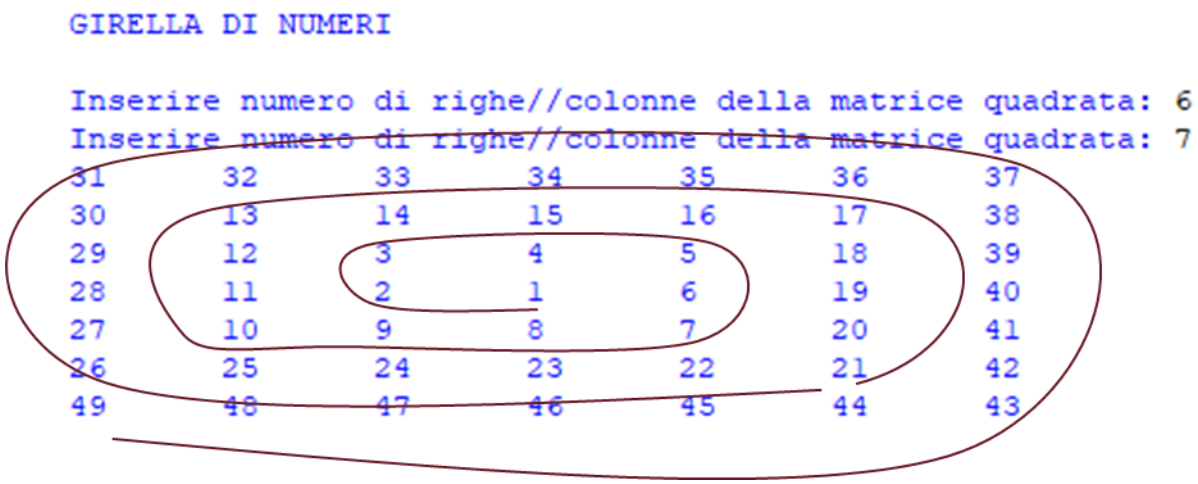
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
GIRELLA DI NUMERI

Inserire numero di righe//colonne della matrice quadrata: 6
Inserire numero di righe//colonne della matrice quadrata: 7
31      32      33      34      35      36      37
30      13      14      15      16      17      38
29      12      3       4       5       18      39
28      11      2       1       6       19      40
27      10      9       8       7       20      41
26      25      24      23      22      21      42
49      48      47      46      45      44      43
```

La figura successiva riprende quella precedente, ma evidenzia la successione con cui sono stati inseriti i numeri da 1 a 49 (7x7 elementi totali nella matrice).



PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).